Organização conceitual da libc

A libc tem duas "estruturas": a **conceitual**, definida pelos padrões (ISO C, POSIX), e a **física**, que cada implementação (glibc, musl, newlib) organiza à sua maneira.

1. Organização conceitual (por área funcional)

Cada cabeçalho padrão corresponde a um módulo:

Área	Cabeçalhos	Conteúdo
Strings e memória	`string.h`	`memcpy`, `strlen`, `strcmp`...
Classificação de caracteres	`ctype.h`, `wctype.h`	`isalpha`, `toupper`...
E/S	`stdio.h`	`FILE`, `printf`, `fopen`, buffering
Utilitários gerais	`stdlib.h`	`malloc`, `qsort`, `atoi`, `exit`, `getenv`
Matemática	`math.h`, `complex.h`, `fenv.h`	normalmente uma biblioteca separada (libm)
Tempo	`time.h`	`time`, `strftime`, fusos
Sinais e saltos	`signal.h`, `setjmp.h`	
Localização	`locale.h`	
Multibyte e wide char	`wchar.h`, `uchar.h`	
Erros e asserções	`errno.h`, `assert.h`	
Argumentos variáveis	`stdarg.h`	
POSIX (extensão)	`unistd.h`, `fcntl.h`, `sys/*`, `pthread.h`, `dirent.h`, `regex.h`...	

2. Organização em camadas

De cima para baixo:

1. **API pública**: funções do padrão.
2. **Implementação portável**: C puro, independente de sistema (`strlen`, `qsort`, `printf` formatador, `malloc` sobre um alocador).
3. **Camada dependente de arquitetura**: versões otimizadas em assembly (`memcpy`, `setjmp`, `longjmp`), ABI, startup code (`crt0`).
4. **Camada de syscalls**: o ponto onde a libc encosta no kernel. É a única parte que muda de verdade de um SO para outro.

3. Estrutura de diretórios em implementações reais

musl (a mais legível, ótima para estudar):

```
musl/
├── include/        # cabeçalhos públicos
├── arch/<cpu>/     # código e bits por arquitetura (syscall_arch.h, bits/)
├── crt/            # crt1.o, crti.o, crtn.o (startup)
├── src/
│   ├── string/  stdio/  stdlib/  math/  time/  ctype/
│   ├── locale/  multibyte/  signal/  setjmp/  errno/
│   ├── malloc/  thread/  process/  unistd/  select/
│   ├── network/  passwd/  regex/  env/  exit/
│   └── internal/   # cabeçalhos privados (stdio_impl.h, syscall.h...)
└── ldso/           # carregador dinâmico
```

glibc (mais complexa, com muita camada de override):

```
glibc/
├── string/ stdio-common/ libio/ stdlib/ math/ time/ ctype/ locale/
├── malloc/ signal/ setjmp/ wcsmbs/ iconv/ nptl/ (threads)
├── nss/ resolv/ inet/ (nomes e rede)
├── csu/            # startup
├── elf/            # linker dinâmico
└── sysdeps/        # o coração da portabilidade
    ├── generic/
    ├── m68k/ x86_64/ arm/ ...        # por arquitetura
    └── unix/sysv/linux/<arch>/       # por SO + arquitetura
```

O `sysdeps/` funciona por **cascata de busca**: o build procura cada arquivo do mais específico (SO+arquitetura) ao mais genérico, e usa o primeiro que encontrar.

newlib (voltada a sistemas embarcados e SOs próprios):

```
newlib/
├── libc/
│   ├── ctype/ stdio/ stdlib/ string/ time/ locale/ signal/ errno/ ...
│   ├── machine/<cpu>/   # assembly por arquitetura
│   ├── reent/           # versões reentrantes, struct _reent
│   ├── syscalls/        # stubs genéricos
│   └── sys/<os>/        # portes para SOs específicos
└── libm/                # matemática separada
```

4. Por que isso importa se você está portando

O desenho da newlib é pensado para esse caso: toda a libc depende de um conjunto mínimo de stubs que você implementa (`_write`, `_read`, `_open`, `_close`, `_lseek`, `_fstat`, `_isatty`, `_sbrk`, `_exit`, `_kill`, `_getpid`). Com isso o `printf`, o `malloc` etc. funcionam sem alterar o resto. Na musl, o equivalente é trocar a camada de `syscall` e o `arch/`.

Se quiser, posso detalhar uma delas (como o `FILE` e o buffering são organizados internamente, como o `malloc` se encaixa, ou o fluxo de startup `crt0` → `main` → `exit`).

libc em detalhe

1. Princípios de organização (valem para todas as implementações)

Uma função por arquivo-fonte. Em musl e newlib, quase cada função (`strlen.c`, `atoi.c`) fica no seu próprio `.c`. Como o `libc.a` é um arquivo de objetos, o linker só puxa os `.o` realmente referenciados, e o binário estático fica pequeno.

Limpeza de espaço de nomes. O ISO C permite que o programa defina uma função chamada `read` ou `index` sem quebrar a libc. Por isso a libc usa nomes internos (`__read`, `_read`, `__libc_*`) e expõe o nome público como *weak alias*:

```
weak_alias(__strlen, strlen);
```

Assim o usuário pode sobrescrever o símbolo sem afetar o uso interno.

Cabeçalhos em três níveis:

- Públicos (`stdio.h`), com macros de teste de recurso (`_POSIX_C_SOURCE`, `_GNU_SOURCE`) que decidem o que fica visível.
- Dependentes de arquitetura (`bits/`, `machine/`): tamanho de `long`, `jmp_buf`, `struct stat`, números de syscall.
- Privados (`stdio_impl.h`, `libc.h`): nunca instalados.

Hosted vs freestanding. O padrão define um subconjunto que não depende de biblioteca: `stddef.h`, `stdint.h`, `stdarg.h`, `limits.h`, `float.h`, `stdbool.h`. Eles vêm do compilador (GCC os instala), não da libc. O resto só existe em ambiente *hosted*.

2. stdio por dentro

Todos os `printf`, `scanf`, `fgets` etc. se apoiam em uma única estrutura, o `FILE`:

- **Buffer**: ponteiros de início, posição de leitura/escrita e fim, mais o tamanho. Há três modos: sem buffer (`stderr`), por linha (terminal) e total (arquivos). O `setvbuf` escolhe o modo.
- **Operações como ponteiros de função**: `read`, `write`, `seek`, `close`, mais um `cookie` (geralmente o descritor). Esse desenho de origem BSD permite que `fopen`, `fmemopen`, `sprintf` e `popen` compartilhem o mesmo motor. Um `sprintf` usa um `FILE` falso cujo "write" copia para a string.
- **Estado**: flags de erro/EOF, modo (leitura/escrita), lista encadeada de todos os `FILE` abertos (para o `exit` dar `fflush` em todos) e um lock por stream (`flockfile`).

- **`ungetc`**: buffer auxiliar pequeno.

`printf` e família são todos finos sobre um único `vfprintf`, que faz o parsing do formato, a conversão de inteiros e a de ponto flutuante. O de ponto flutuante é a parte mais pesada, e a newlib permite desativá-lo (`--enable-newlib-nano-formatted-io`, e `-u _printf_float` para reativar).

No **glibc**, o `libio/` usa uma tabela virtual (`vtable`), e `stdio-common/` guarda o formatador. No **newlib**, `struct __sFILE` é herdada do BSD.

3. Alocação de memória

Implementação	Estratégia
glibc (ptmalloc2)	Arenas por thread; listas segregadas: *tcache*, *fastbins*, *smallbins*, *largebins*, *unsorted*; blocos grandes via `mmap`
musl (mallocng)	Grupos de blocos de mesmo tamanho, com metadados separados e checagens de corrupção
newlib	`malloc` completo (baseado em dlmalloc) ou **nano-malloc** (lista livre simples)

O ponto crucial para portar: todos pedem memória ao sistema por **`brk/sbrk`** e/ou **`mmap`**. Na newlib o contrato é só `_sbrk(incr)`, que devolve um ponteiro para mais memória. Se o SO não tiver `mmap`, é isso que você implementa.

4. Inicialização e término

```
_start (crt1.o / crt0.S)
  ├─ configura a pilha, recebe argc/argv/envp (no Linux, também auxv)
  ├─ zera .bss, inicializa .data (bare-metal)
  ├─ __libc_start_main
  │     ├─ inicializa a libc (TLS, stdio, locale, environ)
  │     ├─ roda construtores (.preinit_array, .init_array)
  │     └─ main(argc, argv, envp)
  └─ exit(retorno)
        ├─ handlers de atexit (ordem inversa do registro)
        ├─ destrutores (.fini_array)
        ├─ fflush em todos os FILE
        └─ _exit(status)   ← única saída real para o kernel
```

Os `crti.o`/`crtn.o` fecham as seções `.init`/`.fini`, e o `crtbegin.o`/`crtend.o` vêm do GCC. Na newlib bare-metal, o `crt0` fica em `libgloss/<arquitetura>/` e chama `__libc_init_array`.

5. A fronteira com o kernel (syscalls)

- **Linux (glibc/musl)**: um macro `syscall` por arquitetura (`arch/<cpu>/syscall_arch.h`). Um retorno entre -4095 e -1 vira `errno` por uma função auxiliar (`__syscall_ret` na musl). Algumas funções são

pontos de cancelamento de threads. O `time` e o `gettimeofday` usam o vDSO, sem entrar no kernel.
- **newlib**: os stubs `_write`, `_read`, `_open`, `_close`, `_lseek`, `_fstat`, `_isatty`, `_sbrk`, `_exit`, `_kill`, `_getpid`. O `libgloss` traz implementações prontas para várias placas e simuladores.

Quem depende de quê:

Função da libc	O que precisa do SO
`strlen`, `qsort`, `atoi`, `snprintf`	nada
`malloc`	`sbrk` ou `mmap`
`fopen`/`printf`/`fgets`	`open`, `read`, `write`, `close`, `lseek`, `fstat`, `isatty`
`exit` / `abort`	`_exit`, `kill` ou `getpid`
`time`, `clock`	relógio do sistema
`getenv`	só o vetor `environ`
`system`, `popen`	`fork`/`exec`/`wait`
`signal`, `raise`	sinais do kernel, ou emulação

6. `errno`, reentrância e threads

- **Monothread**: `errno` é uma variável global.
- **Com threads**: `errno` vira um macro para `*__errno_location()`, e cada thread tem a sua cópia (via TLS).
- **newlib clássica**: uma `struct _reent` agrupa `errno`, os streams padrão, o estado de `rand`, `strtok`, `asctime` e outros. O ponteiro global `_impure_ptr` aponta para ela. Existem variantes explícitas (`printf_r`, `malloc_r`) e ganchos de lock (`__retarget_lock_*`) para você ligar ao seu escalonador.
- **Threads POSIX**: no musl ficam em `src/thread/`; no glibc, em `nptl/` (fundido ao libc desde a 2.34). A newlib não traz threads: você fornece só os locks.

7. Localização, wide chars e conversões

- **glibc**: locales completos (arquivos compilados, `iconv` com módulos *gconv* carregados dinamicamente). É a parte mais pesada.
- **musl**: praticamente só locale C e UTF-8.
- **newlib**: locale C por padrão, com suporte multibyte opcional (`--enable-newlib-mb`).

`wchar.h` (`wcslen`, `mbrtowc`) repete a API de strings para `wchar_t`, e `wctype.h` repete `ctype.h`.

8. Matemática e suporte do compilador

- A **libm** costuma ser separada (`-lm`). A lineagem musl/newlib deriva de **fdlibm** e FreeBSD *msun*, enquanto a glibc tem implementações próprias com variantes otimizadas.
- Sem FPU, o ponto flutuante vem do **libgcc** (`__adddf3`, `__muldf3`...), e em CPUs sem instruções de multiplicação/divisão de 32 bits (como o 68000) também ficam lá `__mulsi3`, `__divsi3`, `__udivsi3`. Ou seja, a libc depende de `libgcc.a` mesmo que você não perceba.

9. Otimização por arquitetura

- As rotinas de `string.h` (`memcpy`, `memset`, `strlen`) têm versão em C genérico e versões em assembly. O padrão é a versão genérica, que lê uma palavra por vez; só depois entra o assembly.
- O glibc escolhe a variante em tempo de execução com **IFUNC** (SSE/AVX/NEON conforme a CPU). Musl e newlib escolhem em tempo de compilação.
- `setjmp/longjmp` e a troca de contexto são sempre assembly, porque dependem de quais registradores a ABI exige preservar.

10. Ordem sugerida para levantar uma libc em um SO novo

1. **Sem SO**: `string.h`, `ctype.h`, `stdlib.h` (conversões, `qsort`, `bsearch`), `stdarg`.
2. **`malloc`** sobre `_sbrk`.
3. **stdio** sobre `read/write/open/close/lseek`.
4. **Startup**: `crt0`, `exit`, `atexit`.
5. **Tempo**, `errno`, `setjmp`, sinais.
6. **POSIX** (`dirent`, `unistd`, `fcntl`), se o SO for oferecê-los.

11. Qual usar como base

- **newlib**: o caminho mais curto para um SO novo; sem dependência de Linux; arquiteturas e estilo bem documentados.
- **musl**: código mais limpo para ler e aprender; assume um modelo de syscall próximo ao do Linux.
- **Mais simples ainda** (para ler, não necessariamente para usar): PDCLib, dietlibc, avr-libc.

Posso aprofundar em qualquer um destes: o fluxo completo de um `printf` até o `write`, o desenho do `malloc` (blocos, coalescência, fragmentação), o contrato exato dos stubs da newlib, ou o layout dos arquivos de um `crt0` mínimo.

Sim, é o arranjo padrão. Elas são complementares: a `libc.a` da newlib implementa as funções do padrão C, e a `libgcc.a` fornece as rotinas auxiliares que o próprio compilador gera chamadas para elas. Uma não substitui a outra, e o código da newlib já vem compilado supondo que a `libgcc` estará lá.

O que a libgcc cobre no 68k

O 68000 não tem multiplicação nem divisão de 32 bits (só 16×16→32 e 32÷16), nem FPU. Então, quando o GCC encontra `a * b` ou `a / b` com `long`, ele emite chamadas como:

- `__mulsi3`, `__divsi3`, `__udivsi3`, `__modsi3`, `__umodsi3` (inteiros de 32 bits)
- `__muldi3`, `__divdi3` (64 bits)
- `__adddf3`, `__muldf3`, `__fixdfsi`... (ponto flutuante por software)

Essas rotinas estão na `libgcc.a`. No 68020 ou superior, as instruções existem em hardware e o GCC as usa direto, com muito menos dependência da `libgcc`.

O ponto mais importante: multilib

A `libc.a` e a `libgcc.a` precisam ser da **mesma variante de CPU** que o seu código (`-m68000`, `-m68020`, `-mcpu32`, `-m68040`, ColdFire). Misturar variantes causa desde instruções inválidas até falhas sutis de ABI. Um toolchain `m68k-elf` com newlib normalmente instala uma cópia de cada biblioteca por variante em subdiretórios. Para conferir:

```
m68k-elf-gcc -print-multi-lib
m68k-elf-gcc -m68000 -print-file-name=libc.a
m68k-elf-gcc -m68000 -print-libgcc-file-name
```

Se os caminhos mostrarem o subdiretório da variante certa, está correto.

Origem do toolchain

Precisa ser um alvo **bare-metal** (`m68k-elf` ou `m68k-unknown-elf`) construído com newlib. Os pacotes `gcc-m68k-linux-gnu` de distribuições Linux usam glibc e não servem para o seu caso. Normalmente você constrói o toolchain com crosstool-NG ou diretamente a partir do binutils + GCC + newlib (`--target=m68k-elf --with-newlib`), escolhendo as variantes no `--with-cpu`/multilib.

Como fica o link

```
m68k-elf-gcc -m68000 -nostartfiles -T orion.ld \
    crt0.o main.o -o app.elf -lc -lgcc
```

Pontos de atenção:

- **Ordem**: `-lc` antes de `-lgcc`, porque a libc é quem puxa símbolos da libgcc. Se houver dependência cruzada, use `-Wl,--start-group -lc -lgcc -Wl,--end-group`.
- **Matemática**: `-lm` também, se usar `math.h`.
- **Stubs do SO**: a newlib deixa indefinidos `_write`, `_read`, `_sbrk` etc. Você tem duas opções: usar `-lnosys` (stubs que só retornam erro, bom para o link passar) ou fornecer os seus, ligados às chamadas do seu sistema. Para o seu SO, a segunda é o objetivo final.
- **`crt0`**: o `libgloss/m68k` traz exemplos para placas específicas, mas num SO próprio você provavelmente escreve o seu (pilha, `.bss`, `.data`, chamada a `__libc_init_array` e depois `main`).
- **Linker script**: precisa definir `_end` (usado pelo `_sbrk` como início do heap) e os símbolos de início/fim de `.bss`/`.data`.

Verificação rápida de que está tudo resolvido

```
m68k-elf-nm -u app.o            # símbolos que seu código ainda precisa
m68k-elf-gcc ... -Wl,-Map=app.map   # mostra de qual .a veio cada símbolo
```

Se aparecer `undefined reference to __mulsi3` ou similar, a `libgcc` não entrou no link (ou é de outra variante). Se aparecer `undefined reference to _sbrk`/`_write`, faltam os stubs do SO.

Se quiser, posso escrever um `crt0.S` mínimo para 68k, um linker script de exemplo e os stubs `_sbrk`/`_write`/`_read` ligados a um console serial.

Onde obter a newlib

Fontes oficiais

- **Página do projeto**: https://sourceware.org/newlib/
- **Tarballs de releases** (o mais simples para começar): https://sourceware.org/pub/newlib/index.html. Pegue o `newlib-X.Y.Z.tar.gz` mais recente. Eu não verifiquei qual é a versão atual, então confira o número no diretório.

- **Git oficial** (versão em desenvolvimento): `git clone https://sourceware.org/git/newlib-cygwin.git`. O repositório é compartilhado com o Cygwin, e a newlib fica no subdiretório `newlib/`, junto com `libgloss/`.

Para um SO próprio, use o **tarball de release**, que é estável e já traz o `configure` pronto no topo. O git de desenvolvimento muda mais e mistura o código do Cygwin.

Compilando para m68k

Você precisa antes de um `m68k-elf-gcc` bare-metal (binutils + GCC já construídos, sem libc). Depois:

```
tar xf newlib-X.Y.Z.tar.gz
mkdir build-newlib && cd build-newlib

../newlib-X.Y.Z/configure \
    --target=m68k-elf \
    --prefix=$HOME/orion-sysroot \
    --disable-newlib-supplied-syscalls \
    --enable-newlib-reent-small \
    --enable-newlib-nano-malloc \
    --disable-newlib-multithread \
    --disable-nls

make -j4
make install
make install DESTDIR=<projeto orion>
```

```
# Estática (orioncore e kernel): endereço fixo, sem PIC
mkdir build-static && cd build-static
../newlib-X.Y.Z/configure --target=m68k-elf \
--prefix=$HOME/orion-sysroot/static \
--disable-newlib-supplied-syscalls
--enable-newlib-nano-malloc \
--enable-newlib-reent-small
--disable-newlib-multithread
--disable-nls \
CFLAGS_FOR_TARGET="-Os -m68000 -ffunction-sections -fdata-sections"
make -j4 && make install DESTDIR=<dir>

# PIC (programas de usuário): mesmas opções, outro CFLAGS e outro prefixo
mkdir ../build-pic && cd ../build-pic
../newlib-X.Y.Z/configure --target=m68k-elf \
--prefix=$HOME/orion-sysroot/pic \
--disable-newlib-supplied-syscalls
--enable-newlib-nano-malloc \
--enable-newlib-reent-small
--disable-newlib-multithread
--disable-nls \
CFLAGS_FOR_TARGET="-Os -m68000 -msep-data -ffunction-sections -fdata-sections"
make -j4 && make install DESTDIR=<dir>
```

O que cada opção faz:

- `--disable-newlib-supplied-syscalls`: não embute os stubs de syscall da própria newlib, para você fornecer os do Orion.
- `--enable-newlib-reent-small`: `struct _reent` menor, importante em máquina com pouca RAM.
- `--enable-newlib-nano-malloc`: `malloc` simples, bem menor que o dlmalloc completo.
- `--disable-newlib-multithread`: sem locks, se o Orion for monotarefa ou você ainda não precisa disso.
- Para restringir a uma CPU só, em vez de todas as variantes: `--disable-multilib` e `CC_FOR_TARGET="m68k-elf-gcc -m68000"` (ajuste o `-m`).
- Se quiser menos código no `printf`/`scanf`, existe também `--enable-newlib-nano-formatted-io`.

O resultado fica em `$HOME/orion-sysroot/m68k-elf/` com `include/` e `lib/` (`libc.a`, `libm.a`, e `crt0.o` se for o da libgloss).

Alternativa: construir tudo junto com o GCC

Se ainda não tem o toolchain, o caminho comum é montar a árvore com binutils, GCC e newlib juntos (`--with-newlib`) ou usar o **crosstool-NG**, que automatiza isso para `m68k-unknown-elf` com newlib e já gera o multilib certo.

Integrando ao Orion

Como o seu SO tem sistema de arquivos e chamadas próprias, você provavelmente quer **adicionar o seu porte** ao invés de depender do `libgloss`:

1. Escreva `orion_syscalls.c` com `_write`, `_read`, `_open`, `_close`, `_lseek`, `_fstat`, `_isatty`, `_sbrk`, `_exit`, `_kill` e `_getpid`, chamando o seu VFS e driver de console.
2. Escreva o seu `crt0.S` e o linker script (com `_end` definido para o `_sbrk`).
3. Linke com `-lc -lgcc`, como vimos antes.

Fazer isso de fora da árvore da newlib (só linkando seus stubs) é o mais fácil de manter, porque você não precisa reconstruir a libc a cada mudança do Orion. Só vale criar um diretório `libc/sys/orion/` dentro da newlib se quiser que ela seja realmente "oficialmente" portada, com `configure.host` próprio.

Opções mais simples, só para estudar

Se o código da newlib for grande demais para ler:

- **PDCLib** (https://github.com/DevSolar/pdclib): C99 limpa, feita justamente para ser portada a SOs novos, com uma camada de plataforma mínima.
- **musl**: legível, mas assume um modelo de syscall parecido com o do Linux.
- **dietlibc / avr-libc**: pequenas, boas de ler.

Posso escrever o conjunto inicial para o Orion (`crt0.S`, linker script e `orion_syscalls.c` com `_sbrk`, `_write` e `_read`) se me disser como é a convenção de chamada do sistema (por exemplo, `trap #n` com número de função em `d0`).

Ótimo, isso é o resultado esperado de uma build da newlib.

O que são os três arquivos

- `libc.a`: a libc em si (string, stdio, stdlib, ctype, time...).
- `libm.a`: a matemática, linkada com `-lm`.
- `libg.a`: na newlib, é só uma **cópia/link da libc.a**. Existe por tradição, porque specs antigos do GCC trocavam `-lc` por `-lg` quando se compilava com `-g`. Hoje não é necessária, e você pode ignorá-la ou apagá-la.

Não sei o que é o "krak" no seu fluxo (um script seu, um gerador de toolchain?), mas o importante é conferir o que ele produziu.

Checagens rápidas

1. **Variante de CPU correta.** Veja se a lib é mesmo para o 68000 (ou o que você escolheu):

   ```
   m68k-elf-objdump -f libc.a | head -20
   m68k-elf-readelf -A libc.a | head
   ```

   Se o toolchain gerou multilib, haverá vários subdiretórios (`m68020/`, `m5200/` etc.) e você deve usar o da sua CPU.

2. **Quais símbolos ficaram indefinidos** (devem ser só os stubs do Orion e as rotinas da libgcc):

   ```
   m68k-elf-nm -u libc.a | sort -u
   ```

   Espere ver `_write`, `_read`, `_open`, `_close`, `_lseek`, `_fstat`, `_isatty`, `_sbrk`, `_exit`, `_kill`, `_getpid`, e coisas como `__mulsi3`/`__divsi3` (da libgcc).

3. **Teste de link mínimo:**

   ```
   m68k-elf-gcc -m68000 -nostartfiles -T orion.ld crt0.o teste.o \
       -o teste.elf -L<dir-das-libs> -lc -lm -lgcc
   ```

   Usando `-Wl,-Map=teste.map` você vê de qual `.a` veio cada símbolo. Os erros de `undefined reference` que sobrarem dizem exatamente o que falta implementar.

Próximo passo

Escrever `orion_syscalls.c`, o `crt0.S` e o linker script (com `_end` para o `_sbrk`). Se me disser como o Orion recebe chamadas de sistema (por exemplo `trap #0` com o número da função em `d0` e argumentos em `d1`-`d3`), eu escrevo esses três arquivos para você.

OrionCore e o kernel rodam em endereço fixo, e os programas carregados pelo kernel são PIC com `-msep-data`, usando A5 e GOT. Se for outra a divisão, me corrija. Com a newlib a divisão continua a mesma, e você faz **duas builds da mesma newlib**.

1. Duas builds, dois prefixos

Os cabeçalhos são idênticos nas duas, então só as bibliotecas mudam.

O `-msep-data` já implica `-fPIC`. Cada árvore terá seu `lib/libc.a` e `lib/libm.a`. Você escolhe a variante com `-L`, sem precisar renomear para `libc68k_pic.a`.

2. O que muda em relação à sua libc própria

Ponteiros em dados inicializados. A newlib tem muito mais dados estáticos com ponteiros do que a sua libc: `_impure_data`, tabelas de locale, `__sglue`, ponteiros para funções. Em PIC eles ficam em `.data` ou `.data.rel.ro` com valor absoluto, e **só o patch da GOT não os corrige**. O sintoma típico é travar no primeiro `printf`, `fopen` ou `malloc`.

A correção você já tem. Linke os programas com `-Wl,-q` (`--emit-relocs`) e use o `apply_data_relocations` (`.rela.data`, `R_68K_32`) do elfload que você já validou. Para checar se há ponteiros assim no binário:

sh

```
m68k-elf-readelf -r app.elf | grep -c R_68K_32
```

`_sbrk`. No programa PIC, `_end` vem pela GOT e já chega como endereço absoluto depois do patch. O `_sbrk` deve limitar o heap ao slot de 256K, entre `_end` e o topo da pilha da task, e devolver erro ao passar disso.

`libgcc.a`. Pode usar a mesma nas duas builds. As rotinas de inteiro (`__mulsi3`, `__divsi3`...) são pc-relativas. Para conferir que não há referência absoluta a dados:

sh

```
m68k-elf-objdump -dr $(m68k-elf-gcc -m68000 -print-libgcc-file-name) | grep R_68K_32
```

Se só aparecer em objetos de ponto flutuante que você não usa, ignore.

3. Reentrância

Como cada programa PIC é linkado estaticamente e tem seu `.data`/`.bss` no próprio slot, ele tem a sua `_impure_data`, o seu `errno` e o seu heap. **Você não precisa trocar `_impure_ptr` no `ctx_switch`**, desde que cada programa seja single-thread. Só vai precisar disso quando implementar threads dentro de um mesmo processo.

4. E no kernel

Eu não usaria o stdio nem o `malloc` da newlib no kernel:

- O `printf` completo é pesado e não é reentrante entre tasks sem locks.

- Heap compartilhado entre tasks exigiria `__malloc_lock`/`__malloc_unlock` (que você poderia implementar desabilitando interrupções, como decidiu para o disco).
- O kernel já tem o VFS e chama `vfs_open`/`vfs_write` diretamente, não precisa dos stubs `_open`/`_write`.

Use a build estática só para o que for útil e sem estado: `string.h`, `ctype.h`, `stdlib.h` (conversões, `qsort`), e escreva um `kprintf` pequeno. O que sobra dos stubs (`_write`, `_read`, `_open`...) só existe na build PIC, onde cada um vira um `trap` com o número em D0.

5. Link de cada lado

sh

```
# kernel/orioncore
m68k-elf-gcc -m68000 -nostartfiles -T kernel.ld ... \
      -L$HOME/orion-sysroot/static/m68k-elf/lib -lc -lgcc

# programa de usuário
m68k-elf-gcc -m68000 -msep-data -nostartfiles -T user.ld -Wl,-q ... \
      -L$HOME/orion-sysroot/pic/m68k-elf/lib -lc -lm -lgcc
```

Se quiser, escrevo os stubs da build PIC (`_write`, `_read`, `_open`, `_close`, `_lseek`, `_fstat`, `_isatty`, `_sbrk`, `_exit`) chamando a trap única, para você trocar no lugar da sua libc própria.

```
# Estática (orioncore e kernel): endereço fixo, sem PIC
mkdir build-static && cd build-static
../newlib-X.Y.Z/configure --target=m68k-elf \
--prefix=$HOME/orion-sysroot/static \
--disable-newlib-supplied-syscalls
--enable-newlib-nano-malloc \
--enable-newlib-reent-small
--disable-newlib-multithread
--disable-nls \
CFLAGS_FOR_TARGET="-Os -m68000 -ffunction-sections -fdata-sections"
make -j4 && make install DESTDIR=<dir>

# PIC (programas de usuário): mesmas opções, outro CFLAGS e outro prefixo
mkdir ../build-pic && cd ../build-pic
../newlib-X.Y.Z/configure --target=m68k-elf \
--prefix=$HOME/orion-sysroot/pic \
--disable-newlib-supplied-syscalls
--enable-newlib-nano-malloc \
--enable-newlib-reent-small
--disable-newlib-multithread
--disable-nls \
CFLAGS_FOR_TARGET="-Os -m68000 -msep-data -ffunction-sections -fdata-sections"
make -j4 && make install DESTDIR=<dir>
```